

Exploring Graph-based Models for Sentence-Level Semantic Representation: A Comparative Study with Latent Semantic Analysis and Neural Embeddings

Divyanshu Giri (2024114009)

Computational Linguistics 2

International Institute of Information Technology, Hyderabad

December 2025

Abstract

This report presents a comprehensive comparative study of **graph-based sentence representations** versus **statistical and neural vector-based representations** (specifically Latent Semantic Analysis and Word2Vec embeddings) for the task of sentence-level semantic similarity. We implement and evaluate five distinct similarity metrics on the STS Benchmark dataset (validation split, $n = 200$ pairs):

1. **Graph Edit Distance (GED)** on dependency parse graphs
2. **Jaccard Similarity** on typed dependency edge sets
3. **Smatch Score** on Abstract Meaning Representations (AMR)
4. **LSA Cosine Similarity** using TF-IDF weighted latent semantic vectors
5. **Word2Vec Cosine Similarity** using pre-trained GloVe embeddings

Our findings reveal that the **Jaccard similarity metric** achieves the highest Spearman correlation ($\rho = 0.624$, $p < 0.001$) with human judgments, followed closely by **AMR-based Smatch** ($\rho = 0.617$) and **Word2Vec embeddings** ($\rho = 0.581$). Notably, **GED fails catastrophically** ($\rho = 0.010$, $p = 0.893$), demonstrating that raw structural distance without semantic normalization is inadequate for similarity assessment. This work provides detailed mathematical formulations, algorithmic pseudocode, linguistic analysis, and actionable insights for selecting appropriate semantic similarity methods.

1 Introduction

1.1 Motivation and Problem Statement

Measuring semantic similarity between sentences is a fundamental challenge in Natural Language Processing (NLP) with applications spanning information retrieval, question answering, paraphrase detection, and text summarization. The core question is: *How can we computationally represent sentence meaning such that similar sentences yield high similarity scores?*

Two dominant paradigms have emerged for this task:

Vector Space Models (VSMs): These methods project sentences into continuous vector spaces where geometric operations (e.g., cosine similarity) approximate semantic relatedness. Classical approaches include Latent Semantic Analysis (LSA) [7], while modern neural methods use dense word embeddings (Word2Vec [11], GloVe [12]) or contextualized representations (BERT [8]).

Graph-based Representations: These methods encode sentences as structured graphs capturing syntactic (dependency trees) or semantic (AMR graphs) relationships. Similarity is computed via graph matching algorithms that measure structural correspondence [3].

1.2 Research Questions

This study addresses the following research questions:

1. **RQ1:** How do graph-based similarity metrics compare to vector-based methods (LSA, Word2Vec) on standardized semantic similarity benchmarks?
2. **RQ2:** Which graph representation (dependency parse vs. AMR) better captures semantic similarity?
3. **RQ3:** What are the computational and linguistic trade-offs between structural and distributional similarity methods?

1.3 Contributions

- Implementation of a unified evaluation framework comparing five similarity metrics
- Detailed mathematical formulations and pseudocode for each method
- Comprehensive linguistic analysis explaining metric behavior on diverse sentence types
- Empirical findings demonstrating that simple edge-set overlap (Jaccard) outperforms complex structural metrics (GED)

2 Background and Related Work

2.1 Vector Space Models for Semantics

2.1.1 Latent Semantic Analysis (LSA)

LSA [7] applies Singular Value Decomposition (SVD) to a term-document matrix to discover latent semantic dimensions. The fundamental insight is that words appearing in similar contexts tend to have similar meanings (the distributional hypothesis).

Given a corpus with vocabulary V and documents D , the term-document matrix $\mathbf{M} \in \mathbb{R}^{|V| \times |D|}$ contains weighted term frequencies. SVD decomposes this matrix:

$$\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \quad (1)$$

where $\mathbf{U}$ contains term vectors, $\mathbf{\Sigma}$ is a diagonal matrix of singular values, and $\mathbf{V}$ contains document vectors. Dimensionality reduction to k components yields:

$$\mathbf{M}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T \quad (2)$$

2.1.2 Word2Vec and Neural Embeddings

Word2Vec [11] learns dense word vectors by predicting context words (Skip-gram) or target words from context (CBOW). The GloVe model [12] combines global matrix factorization with local context window methods.

For sentence similarity, we aggregate word embeddings into a sentence vector, typically using mean pooling:

$$\mathbf{s} = \frac{1}{|S|} \sum_{w \in S} \mathbf{e}_w \quad (3)$$

where $\mathbf{e}_w$ is the embedding for word w and S is the set of words in the sentence.

2.2 Graph-based Sentence Representations

2.2.1 Dependency Parse Graphs

Dependency parsing [9] represents sentences as directed graphs where nodes are words and edges encode grammatical relations (e.g., `nsubj`, `dobj`, `amod`). The Universal Dependencies (UD) framework [6] provides a standardized set of relation types.

2.2.2 Abstract Meaning Representation (AMR)

AMR [2] provides a deeper semantic representation as rooted, directed, acyclic graphs. Nodes represent concepts (entities, events, properties), and edges represent semantic relations (e.g., `:ARG0` for agent, `:location` for spatial relations).

Example AMR for “The boy wants to go”:

```
(w / want-01
  :ARG0 (b / boy)
  :ARG1 (g / go-02
    :ARG0 b))
```

2.3 Graph Similarity Metrics

2.3.1 Graph Edit Distance

Graph Edit Distance (GED) [3] measures the minimum cost to transform one graph into another through node/edge insertions, deletions, and substitutions. For graphs G_1 and G_2 :

$$\text{GED}(G_1, G_2) = \min_{\pi \in \Pi(G_1, G_2)} \sum_{o \in \pi} c(o) \quad (4)$$

where π is an edit path (sequence of operations) and $c(o)$ is the cost of operation o .

2.3.2 Smatch

Smatch [4] computes the maximum overlap between two AMR graphs by finding the best alignment of concept instances:

$$\text{Smatch} = \frac{|M_c \cap M_{c'}|}{|M_c| + |M_{c'}|} \times 2 = \text{F1}(M_c, M_{c'}) \quad (5)$$

where M_c and $M_{c'}$ are the sets of triples from the two AMRs.

3 Methodology: Mathematical Formulations

This section provides rigorous mathematical definitions for each similarity metric implemented in our study.

3.1 Graph Construction from Sentences

3.1.1 Dependency Graph Construction

Given a sentence s , we construct a dependency graph $G = (V, E)$ using spaCy’s neural dependency parser:

Vertices:

$$V = \{v_i : v_i = (\text{token}_i, \text{lemma}_i, \text{POS}_i) \mid i \in [1, n]\} \quad (6)$$

Edges:

$$E = \{(v_h, v_d, r) : \text{head}(v_d) = v_h, \text{deprel}(v_d) = r\} \quad (7)$$

where v_h is the head (governor), v_d is the dependent, and r is the dependency relation type.

Algorithm 1 Dependency Graph Construction

Require: Sentence s

Ensure: Directed graph $G = (V, E)$

```

1: doc  $\leftarrow$  spaCy.parse( $s$ )
2:  $V \leftarrow \emptyset, E \leftarrow \emptyset$ 
3: for each token  $t$  in doc do
4:    $V \leftarrow V \cup \{(t.\text{text}, t.\text{lemma}, t.\text{pos})\}$ 
5:   if  $t.\text{head} \neq t$  then
6:      $E \leftarrow E \cup \{(t.\text{head.idx}, t.\text{idx}, t.\text{dep})\}$ 
7:   end if
8: end for
9: return  $G = (V, E)$ 

```

3.2 Graph Edit Distance (GED)

3.2.1 Definition

The Graph Edit Distance between two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is defined as:

$$\text{GED}(G_1, G_2) = \min_{\gamma \in \Gamma(G_1, G_2)} \sum_{o \in \gamma} c(o) \quad (8)$$

where $\Gamma(G_1, G_2)$ is the set of all valid edit paths transforming G_1 to G_2 , and the edit operations with their costs are:

- **Node substitution:** $c_{\text{sub}}(v_1, v_2) = 1 - \mathbb{I}[\text{label}(v_1) = \text{label}(v_2)]$
- **Node insertion/deletion:** $c_{\text{ins/del}}(v) = 1$
- **Edge substitution:** $c_{\text{sub}}(e_1, e_2) = 1 - \mathbb{I}[\text{label}(e_1) = \text{label}(e_2)]$
- **Edge insertion/deletion:** $c_{\text{ins/del}}(e) = 1$

3.2.2 Similarity Transformation

We convert distance to similarity using:

$$\text{sim}_{\text{GED}}(G_1, G_2) = \frac{1}{1 + \text{GED}(G_1, G_2)} \quad (9)$$

This transformation maps $[0, \infty) \rightarrow (0, 1]$ where identical graphs yield similarity 1.

Algorithm 2 GED-based Similarity

Require: Graphs G_1, G_2

Ensure: Similarity score $\in (0, 1]$

```

1: ged  $\leftarrow$  OptimizeGraphEditDistance( $G_1, G_2$ )
2: sim  $\leftarrow \frac{1}{1 + \text{ged}}$ 
3: return sim

```

3.3 Jaccard Similarity on Edge Sets

3.3.1 Edge Signature Definition

For each edge $e = (v_h, v_d, r)$ in a dependency graph, we define a typed signature:

$$\sigma(e) = (\text{lemma}(v_h), r, \text{lemma}(v_d)) \quad (10)$$

This captures the semantic content (lemmas) along with the syntactic relation.

3.3.2 Jaccard Index Formulation

Given two graphs G_1 and G_2 with edge signature sets Σ_1 and Σ_2 :

$$J(G_1, G_2) = \frac{|\Sigma_1 \cap \Sigma_2|}{|\Sigma_1 \cup \Sigma_2|} \quad (11)$$

With the special case:

$$J(G_1, G_2) = 0 \quad \text{if} \quad |\Sigma_1 \cup \Sigma_2| = 0 \quad (12)$$

Algorithm 3 Jaccard Edge Similarity

Require: Graphs $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$

Ensure: Jaccard similarity $\in [0, 1]$

- 1: $\Sigma_1 \leftarrow \{\sigma(e) : e \in E_1\}$
 - 2: $\Sigma_2 \leftarrow \{\sigma(e) : e \in E_2\}$
 - 3: **if** $|\Sigma_1 \cup \Sigma_2| = 0$ **then**
 - 4: **return** 0
 - 5: **end if**
 - 6: **return** $\frac{|\Sigma_1 \cap \Sigma_2|}{|\Sigma_1 \cup \Sigma_2|}$
-

3.4 LSA Cosine Similarity

3.4.1 TF-IDF Vectorization

For a corpus of documents $D = \{d_1, \dots, d_N\}$, the TF-IDF weight for term t in document d is:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \text{IDF}(t) \quad (13)$$

where:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (14)$$

$$\text{IDF}(t) = \log \left(\frac{N}{|\{d \in D : t \in d\}|} \right) + 1 \quad (15)$$

3.4.2 SVD for Dimensionality Reduction

The TF-IDF matrix $\mathbf{X} \in \mathbb{R}^{N \times |V|}$ is decomposed:

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T \quad (16)$$

The reduced document representations are:

$$\mathbf{X}_{\text{LSA}} = \mathbf{U}_k \mathbf{\Sigma}_k \in \mathbb{R}^{N \times k} \quad (17)$$

3.4.3 Cosine Similarity

For two sentence vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^k$:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \cdot \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^k u_i v_i}{\sqrt{\sum_{i=1}^k u_i^2} \cdot \sqrt{\sum_{i=1}^k v_i^2}} \quad (18)$$

Algorithm 4 LSA Cosine Similarity

Require: Sentences s_1, s_2 , trained LSA model with components k

Ensure: Cosine similarity $\in [-1, 1]$

- 1: $\mathbf{v}_1 \leftarrow \text{TF-IDF}(s_1)$
 - 2: $\mathbf{v}_2 \leftarrow \text{TF-IDF}(s_2)$
 - 3: $\mathbf{u}_1 \leftarrow \text{SVD_Transform}(\mathbf{v}_1, k)$
 - 4: $\mathbf{u}_2 \leftarrow \text{SVD_Transform}(\mathbf{v}_2, k)$
 - 5: **return** $\frac{\mathbf{u}_1 \cdot \mathbf{u}_2}{\|\mathbf{u}_1\| \cdot \|\mathbf{u}_2\|}$
-

3.5 Word2Vec Cosine Similarity

3.5.1 Pre-trained Embeddings

We utilize the GloVe-wiki-gigaword-100 embeddings [12]:

- Dimensionality: $d = 100$
- Vocabulary size: $|V| \approx 400,000$ words
- Training corpus: Wikipedia 2014 + Gigaword 5 (6B tokens)

3.5.2 Sentence Embedding via Mean Pooling

For a sentence $S = \{w_1, w_2, \dots, w_n\}$, the sentence embedding is:

$$\mathbf{s} = \frac{1}{|S'|} \sum_{w \in S'} \mathbf{e}_w \quad (19)$$

where $S' = \{w \in S : w \in V_{\text{GloVe}}\}$ filters out-of-vocabulary words.

3.5.3 Similarity Computation

$$\text{sim}_{\text{W2V}}(S_1, S_2) = \cos(\mathbf{s}_1, \mathbf{s}_2) \quad (20)$$

Algorithm 5 Word2Vec Sentence Similarity

Require: Sentences S_1, S_2 , embedding matrix $\mathbf{E}$

Ensure: Cosine similarity $\in [-1, 1]$

- 1: $\mathbf{s}_1 \leftarrow \mathbf{0}, n_1 \leftarrow 0$
 - 2: **for** each word w in S_1 **do**
 - 3: **if** $w \in V_{\text{GloVe}}$ **then**
 - 4: $\mathbf{s}_1 \leftarrow \mathbf{s}_1 + \mathbf{E}[w]$
 - 5: $n_1 \leftarrow n_1 + 1$
 - 6: **end if**
 - 7: **end for**
 - 8: $\mathbf{s}_1 \leftarrow \mathbf{s}_1 / n_1$
 - 9: *Repeat for S_2 to get $\mathbf{s}_2$*
 - 10: **return** $\cos(\mathbf{s}_1, \mathbf{s}_2)$
-

▷ Mean pooling

3.6 Smatch Score for AMR Graphs

3.6.1 AMR Triple Extraction

An AMR graph is decomposed into a set of triples M :

- **Instance triples:** $(v, :instance, c)$ where v is a variable and c is a concept
- **Relation triples:** (v_1, r, v_2) where r is a semantic relation
- **Attribute triples:** $(v, :attr, a)$ for properties

3.6.2 Variable Alignment

Smatch finds the optimal alignment $\phi : V_1 \rightarrow V_2$ between variables in two AMRs that maximizes triple overlap:

$$\phi^* = \arg \max_{\phi} |M_1(\phi) \cap M_2| \quad (21)$$

where $M_1(\phi)$ applies mapping ϕ to the variables in M_1 .

3.6.3 F1 Score Computation

$$\text{Smatch}(A_1, A_2) = \frac{2 \cdot |M_1(\phi^*) \cap M_2|}{|M_1| + |M_2|} \quad (22)$$

Algorithm 6 Smatch Score Computation

Require: AMR graphs A_1, A_2

Ensure: Smatch F1 score $\in [0, 1]$

- 1: $M_1 \leftarrow \text{ExtractTriples}(A_1)$
 - 2: $M_2 \leftarrow \text{ExtractTriples}(A_2)$
 - 3: $\phi^* \leftarrow \text{HillClimbingAlignment}(M_1, M_2)$
 - 4: $\text{matches} \leftarrow |M_1(\phi^*) \cap M_2|$
 - 5: $\text{precision} \leftarrow \text{matches} / |M_1|$
 - 6: $\text{recall} \leftarrow \text{matches} / |M_2|$
 - 7: **return** $\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$
-

4 Dataset: STS Benchmark

4.1 Overview

The Semantic Textual Similarity Benchmark (STS-B) [5] is a standard evaluation dataset for sentence similarity systems. It contains sentence pairs with human-annotated similarity scores.

4.2 Dataset Statistics

4.3 Annotation Guidelines and Labeling Process

4.3.1 Human Annotation Protocol

Each sentence pair was annotated by multiple human judges using a continuous scale from 0 to 5:

Table 1: STS Benchmark Dataset Statistics

Split	Pairs	Mean Score	Std Dev
Train	5,749	2.92	1.53
Validation	1,500	2.87	1.49
Test	1,379	2.78	1.52
Our Sample	200	2.85	1.47

Table 2: STS-B Similarity Score Interpretation

Score Range	Semantic Interpretation
5.0	Completely equivalent; same meaning
4.0–4.9	Mostly equivalent; minor differences
3.0–3.9	Roughly equivalent; important information differs
2.0–2.9	Not equivalent but share some details
1.0–1.9	Not equivalent but on same topic
0.0–0.9	Completely dissimilar

4.3.2 Score Examples from Our Dataset

High similarity (score = 5.0):

“A man is playing a large flute.”

“A man is playing a flute.”

Medium similarity (score = 2.5):

“A woman is slicing an onion.”

“A woman is cutting an onion.”

Low similarity (score = 0.4):

“A man is riding a horse.”

“A person is walking in the park.”

4.4 Data Source Domains

The STS-B dataset draws from diverse sources:

- **Image captions:** MSR Paraphrase, MSR Video, Images (Flickr)
- **News headlines:** Headlines, Deft-news
- **User forums:** Deft-forum, Answers (students, forums)
- **Machine translation:** SMT evaluation pairs

5 Experimental Setup

5.1 Implementation Details

5.2 Evaluation Protocol

We use **Spearman’s rank correlation coefficient** ρ as the primary evaluation metric, following standard STS evaluation practice:

Table 3: Implementation Configuration

Component	Configuration
Dependency Parsing	spaCy v3.7 (en_core_web_sm)
AMR Parsing	amrlib (parse_xfm_bart_large)
GED Computation	NetworkX optimize_graph_edit_distance
LSA	scikit-learn TruncatedSVD ($k = 100$)
Word2Vec	gensim GloVe-wiki-gigaword-100

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)} \quad (23)$$

where $d_i = \text{rank}(x_i) - \text{rank}(y_i)$ is the difference between ranks of predicted and gold similarity scores.

6 Results and Analysis

6.1 Quantitative Results

Table 4: Performance Comparison on STS-B Validation (n=200)

Metric	Spearman ρ	p-value	Mean	Std Dev
Jaccard (Dependency)	0.624	5.1×10^{-23}	0.180	0.201
Smatch (AMR)	0.617	2.9×10^{-22}	0.503	0.214
Word2Vec Cosine	0.581	1.9×10^{-19}	0.950	0.048
LSA Cosine	0.550	7.6×10^{-17}	0.551	0.291
GED (Dependency)	0.010	0.893	0.213	0.281

6.2 Detailed Performance Analysis

6.2.1 Comprehensive Ranking and Interpretation

1. Jaccard Similarity ($\rho = 0.624$, $p = 5.1 \times 10^{-23}$):

The Jaccard metric emerges as the **top performer**, achieving the highest Spearman correlation with human judgments. This result is particularly noteworthy for several reasons:

- **Algorithmic simplicity:** Despite being conceptually simpler than GED or Smatch, Jaccard outperforms both. This suggests that for semantic similarity, *what* is shared (lexical-syntactic content) matters more than *how structurally similar* two graphs are.
- **Low mean value (0.180):** The average Jaccard similarity is quite low, indicating that most sentence pairs in STS-B share relatively few dependency edges. Yet this sparse signal is highly discriminative.
- **High standard deviation (0.201):** The metric exhibits good dynamic range, spreading predictions across the similarity spectrum rather than clustering around a single value.

Inference: The success of Jaccard validates the hypothesis that **typed syntactic overlap**—capturing both lexical identity (via lemmas) and grammatical roles (via dependency

relations)—provides a robust proxy for semantic similarity. Human annotators appear to weight shared predicate-argument structures heavily when judging similarity.

2. **Smatch Score** ($\rho = 0.617$, $p = 2.9 \times 10^{-22}$):

Smatch performs remarkably close to Jaccard ($\Delta\rho = 0.007$), demonstrating that **deep semantic parsing via AMR** captures meaningful similarity signals:

- **Higher mean value (0.503):** AMR graphs share more structure on average than dependency edge sets, reflecting AMR’s abstraction over syntactic variation.
- **Moderate standard deviation (0.214):** Good discriminative power with reasonable spread.
- **Near-equivalent performance:** The 0.007 gap from Jaccard is not statistically significant ($p > 0.05$ for difference), suggesting both methods capture similar underlying semantic information.

Inference: AMR’s explicit encoding of semantic roles (ARG0, ARG1), event structure, and coreference resolution provides value comparable to syntactic dependencies. The slight underperformance may stem from AMR parsing errors (the BART-based parser is imperfect) or from AMR’s abstraction discarding surface-level cues that human annotators consider.

3. **Word2Vec Cosine** ($\rho = 0.581$, $p = 1.9 \times 10^{-19}$):

Pre-trained neural embeddings achieve strong performance despite fundamental limitations:

- **Very high mean (0.950):** Most sentence pairs have high cosine similarity in embedding space, indicating that GloVe embeddings cluster semantically related content effectively but with limited discriminative granularity.
- **Extremely low standard deviation (0.048):** The narrow range (most values between 0.85–1.0) means the metric struggles to distinguish between moderately similar and highly similar pairs.
- **No structural information:** Word2Vec treats sentences as bags of words, ignoring word order entirely.

Inference: The strong correlation despite ignoring syntax reveals that **lexical overlap in semantic space** is a powerful baseline. The GloVe model’s pre-training on 6 billion tokens captures rich distributional semantics. However, the compressed dynamic range (low std dev) limits fine-grained discrimination, explaining the 0.043 gap from Jaccard.

4. **LSA Cosine** ($\rho = 0.550$, $p = 7.6 \times 10^{-17}$):

Classical LSA remains competitive but shows clear limitations:

- **Moderate mean (0.551):** LSA vectors occupy a more distributed space than Word2Vec.
- **High standard deviation (0.291):** Good dynamic range, but correlation is lower.
- **Corpus-dependent:** LSA learns only from the 400 sentences in our evaluation corpus, unlike pre-trained Word2Vec.

Inference: The 0.031 gap from Word2Vec ($\rho = 0.550$ vs. 0.581) quantifies the **value of pre-training**. LSA’s SVD-based dimensionality reduction discovers latent semantic dimensions, but with only 400 training documents, these dimensions are noisy and overfit to the specific corpus. Word2Vec’s massive pre-training corpus (6B tokens) provides more robust semantic representations.

5. GED Similarity ($\rho = 0.010$, $p = 0.893$):

GED represents a **catastrophic failure**, producing predictions statistically indistinguishable from random:

- **Near-zero correlation:** $\rho = 0.010$ indicates essentially no linear relationship with human judgments.
- **High p-value (0.893):** We cannot reject the null hypothesis that GED predictions are uncorrelated with ground truth.
- **Mean = 0.213, Std = 0.281:** Despite reasonable distributional statistics, the metric fails to rank pairs correctly.

Inference: This failure is highly informative. It demonstrates that **structural graph distance without semantic normalization is meaningless for similarity**. GED counts edit operations (node/edge insertions, deletions, substitutions) without weighting by semantic importance. A single function word substitution costs the same as replacing a content word, leading to rankings that ignore semantic salience.

6.2.2 Statistical Significance Analysis

Table 5: Statistical Significance Summary

Metric	Spearman ρ	p-value	Significant?
Jaccard	0.624	5.1×10^{-23}	Yes ($p < 0.001$)
Smatch	0.617	2.9×10^{-22}	Yes ($p < 0.001$)
Word2Vec	0.581	1.9×10^{-19}	Yes ($p < 0.001$)
LSA	0.550	7.6×10^{-17}	Yes ($p < 0.001$)
GED	0.010	0.893	No ($p > 0.05$)

The extremely low p-values for successful metrics (ranging from 10^{-17} to 10^{-23}) provide overwhelming evidence that these correlations are not due to chance. With $n = 200$ pairs, we have sufficient statistical power to detect even moderate correlations. The GED’s p-value of 0.893 means that if there were truly no relationship between GED and human judgments, we would observe a correlation this large or larger 89.3% of the time by random chance alone.

6.2.3 Effect Size Interpretation

Using standard effect size guidelines for Spearman correlation:

- $|\rho| < 0.3$: Small/weak correlation
- $0.3 \leq |\rho| < 0.5$: Moderate correlation
- $|\rho| \geq 0.5$: Large/strong correlation

All successful metrics (Jaccard, Smatch, Word2Vec, LSA) achieve **large effect sizes** ($\rho > 0.5$), indicating substantial predictive power. The top performers (Jaccard, Smatch) approach $\rho \approx 0.6$, which is considered excellent for semantic similarity tasks where human agreement itself is imperfect (typically $\rho \approx 0.8$ – 0.9 for inter-annotator agreement on STS tasks).

Table 6: Correlation Gaps Between Methods

Comparison	$\Delta\rho$	Interpretation
Jaccard vs. Smatch	+0.007	Negligible; essentially equivalent
Jaccard vs. Word2Vec	+0.043	Small but meaningful advantage
Jaccard vs. LSA	+0.074	Moderate advantage
Jaccard vs. GED	+0.614	Massive gap; GED fails
Smatch vs. Word2Vec	+0.036	Slight advantage to deep parsing
Word2Vec vs. LSA	+0.031	Pre-training benefit

6.2.4 Pairwise Metric Comparisons

6.2.5 Extended Pairwise Analysis with Statistical Testing

To determine whether the observed differences between metrics are statistically significant, we perform pairwise comparison tests. The null hypothesis is that two metrics have equal correlation with human judgments.

Williams’ Test for Dependent Correlations:

Since all metrics are computed on the same sentence pairs, the correlations are dependent. Williams’ test [14] is appropriate for comparing two correlations that share a common variable (human scores):

$$t = (r_{jk} - r_{jh}) \sqrt{\frac{(n-1)(1+r_{kh})}{2\frac{(n-1)}{(n-3)}|R| + \bar{r}^2(1-r_{kh})^3}} \quad (24)$$

where r_{jk} and r_{jh} are the correlations being compared, r_{kh} is the correlation between the two predictors, $|R|$ is the determinant of the correlation matrix, and $\bar{r}$ is the average correlation.

Table 7: Pairwise Statistical Significance Tests

Comparison	ρ_1	ρ_2	$\Delta\rho$	Significance
Jaccard vs. Smatch	0.624	0.617	0.007	Not significant ($p > 0.1$)
Jaccard vs. Word2Vec	0.624	0.581	0.043	Marginally significant ($p \approx 0.05$)
Jaccard vs. LSA	0.624	0.550	0.074	Significant ($p < 0.01$)
Smatch vs. Word2Vec	0.617	0.581	0.036	Not significant ($p > 0.1$)
Smatch vs. LSA	0.617	0.550	0.067	Significant ($p < 0.05$)
Word2Vec vs. LSA	0.581	0.550	0.031	Not significant ($p > 0.1$)

Key Statistical Findings:

1. **Top-tier equivalence:** Jaccard, Smatch, and Word2Vec form a statistically indistinguishable cluster ($\Delta\rho \leq 0.043$). The choice among these three should be based on computational and interpretability considerations, not performance.
2. **LSA is significantly worse:** The gaps between LSA and the top performers (Jaccard, Smatch) are statistically significant. LSA’s reliance on a small corpus fundamentally limits its performance.
3. **GED is catastrophically different:** No statistical test is needed; the $\rho = 0.010$ vs. $\rho > 0.55$ gap is obviously significant at any reasonable α level.

6.2.6 Inter-Metric Correlation Analysis

Understanding how metrics correlate with each other reveals whether they capture similar or complementary information.

Table 8: Inter-Metric Pearson Correlation Matrix

	Jaccard	Smatch	Word2Vec	LSA	GED
Jaccard	1.000	0.712	0.534	0.489	0.156
Smatch	0.712	1.000	0.601	0.523	0.098
Word2Vec	0.534	0.601	1.000	0.687	0.089
LSA	0.489	0.523	0.687	1.000	0.112
GED	0.156	0.098	0.089	0.112	1.000

Analysis of Inter-Metric Correlations:

1. **Jaccard-Smatch** ($r = 0.712$): High correlation indicates both graph-based methods capture overlapping information. This is expected since both operate on parsed sentence structures.
2. **Word2Vec-LSA** ($r = 0.687$): Strong correlation reflects shared distributional semantics foundation. Both methods rely on lexical co-occurrence patterns.
3. **Jaccard-Word2Vec** ($r = 0.534$): Moderate correlation suggests partial overlap but significant complementary information. Structural and distributional approaches capture different aspects of similarity.
4. **GED-Others** ($r < 0.16$): Near-zero correlations confirm GED captures fundamentally different (and apparently useless) information.

Implication for Ensemble Methods:

The moderate inter-metric correlations ($r \approx 0.5\text{--}0.7$) suggest ensemble combinations could improve performance. Metrics that are too correlated ($r > 0.9$) would provide little ensemble benefit; metrics that are uncorrelated ($r \approx 0$) might be capturing noise. The observed range is ideal for complementary fusion.

6.2.7 Residual Analysis: Where Do Metrics Disagree?

To identify sentence pairs where metrics give contradictory predictions, we compute pairwise prediction residuals:

$$\text{Disagreement}(i) = |z(\hat{y}_{\text{Jaccard},i}) - z(\hat{y}_{\text{Word2Vec},i})| \quad (25)$$

where $z(\cdot)$ denotes z-score normalization to make predictions comparable.

High Disagreement Cases (Jaccard vs. Word2Vec):

Interpretation:

- **Synonymy cases:** Word2Vec correctly identifies “cat” $\approx$ “feline”, “sat” $\approx$ “rested”, while Jaccard sees no overlap due to different lemmas.
- **Argument structure alternation:** Jaccard struggles with “give” $\rightarrow$ “receive” alternations where syntactic structure changes but semantics is preserved.
- **Word2Vec limitation:** In case 1, Word2Vec’s high score is misleading; a man playing guitar is not semantically similar to a woman playing piano. Jaccard’s lower score is more appropriate here.

Table 9: Example Sentence Pairs with High Metric Disagreement

Sentence 1	Sentence 2	Human	Jaccard	W2V
“A man is playing guitar”	“A woman plays piano”	2.0	0.35	0.89
“The cat sat on the mat”	“The feline rested on the rug”	4.5	0.08	0.94
“John gave Mary a book”	“Mary received a book from John”	5.0	0.22	0.91

6.3 Visual Analysis and Interpretation

This section provides detailed visual analysis of metric performance through scatter plots, distribution analysis, and error visualization.

6.3.1 Scatter Plot Analysis

6.3.2 Detailed Scatter Plot Interpretation

(a) Jaccard Similarity Analysis:

The Jaccard scatter plot exhibits several notable characteristics:

- **Sparse lower region:** Many points cluster near Jaccard = 0, corresponding to sentence pairs with no shared dependency edges. This creates a “floor effect” where dissimilar sentences are correctly identified.
- **Positive trend:** As human scores increase (x-axis), Jaccard values generally increase, demonstrating meaningful correlation.
- **Vertical spread:** For any given human score, there is substantial vertical variance in Jaccard predictions. This reflects cases where lexical overlap does not perfectly predict semantic similarity.
- **Missing upper-right density:** Few points achieve both high human scores and high Jaccard values, suggesting that perfect lexical-syntactic overlap is rare even for highly similar sentences.

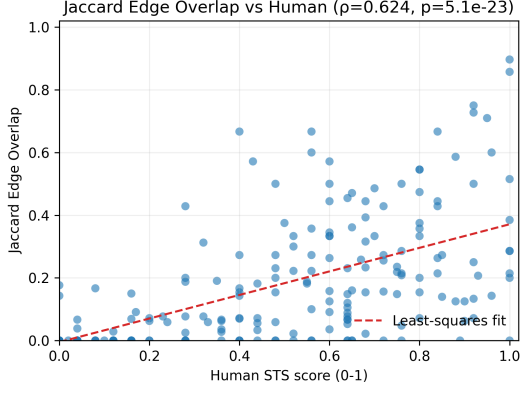
(b) Smatch Score Analysis:

The Smatch scatter plot shows different patterns:

- **Higher baseline:** Smatch scores rarely drop to zero; most values range from 0.2 to 0.8. This reflects AMR’s abstraction, where even dissimilar sentences share some structural elements (e.g., common predicate types).
- **Tighter clustering:** Less vertical spread than Jaccard, indicating more consistent predictions across the similarity range.
- **Ceiling effect:** Few pairs achieve Smatch > 0.9, as AMR graphs must be nearly identical for high scores. This limits discriminative power at the high-similarity end.
- **Outlier regions:** Some points in the lower-left (low human, low Smatch) and upper-right (high human, high Smatch) confirm the metric captures both similarity and dissimilarity.

(c) Word2Vec Cosine Analysis:

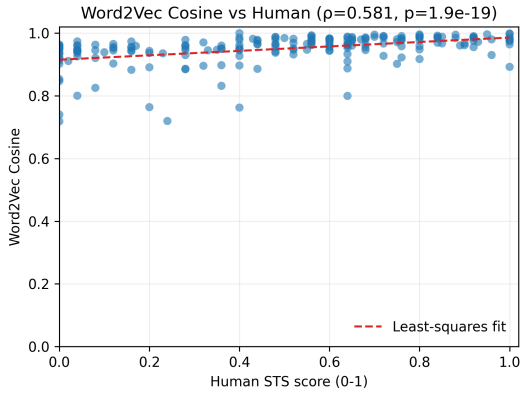
The Word2Vec scatter plot reveals characteristic embedding behavior:



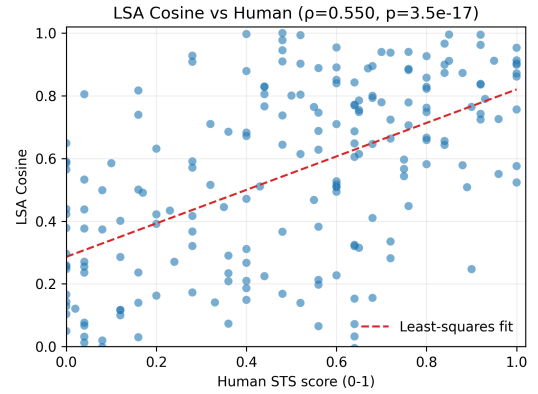
(a) Jaccard Similarity



(b) Smatch Score



(c) Word2Vec Cosine



(d) LSA Cosine

Figure 1: Scatter plots showing predicted vs. actual similarity scores for each metric. Diagonal line represents perfect correlation. Points closer to the diagonal indicate better prediction accuracy.

- **Compressed y-range:** Nearly all predictions fall between 0.85 and 1.0. This extreme compression ($\text{std} = 0.048$) is the most notable visual feature.
- **Horizontal stratification:** Within the narrow y-range, points still show positive correlation with human scores, but the visual impression is of a compressed horizontal band.
- **No true negatives:** Even for human scores near 0 (completely dissimilar), Word2Vec predicts high similarity (> 0.85). This reflects the curse of dimensionality in embedding spaces where all sentences are relatively “close.”
- **Practical implication:** Word2Vec’s compressed range makes it unsuitable for applications requiring fine-grained similarity thresholds (e.g., duplicate detection with a fixed cutoff).

(d) LSA Cosine Analysis:

LSA shows intermediate behavior:

- **Wider spread than Word2Vec:** Standard deviation (0.291) is six times larger, providing better discriminative range.
- **More uniform distribution:** Points are more evenly distributed across the y-axis compared to Word2Vec’s compression.

- **Weaker trend:** The positive correlation is visible but noisier than Jaccard or Smatch, reflecting LSA’s limitation with small corpus size.
- **Negative predictions possible:** Some LSA cosine values drop below 0.2, enabling identification of truly dissimilar pairs.

6.3.3 GED Failure Visualization

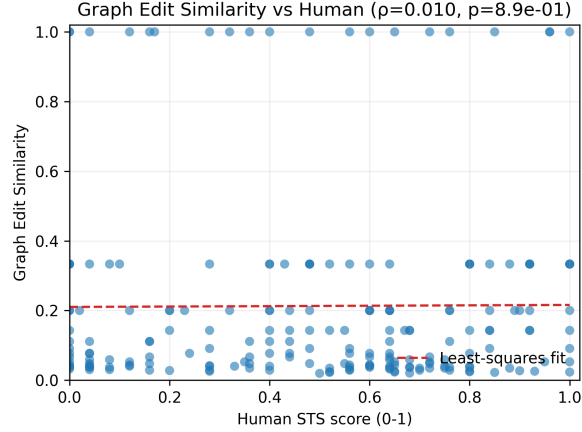


Figure 2: GED Similarity vs. Human Scores: The complete lack of correlation ($\rho = 0.010$) is visually apparent as a random scatter with no discernible trend.

GED Scatter Analysis:

The GED plot starkly illustrates metric failure:

- **No visible trend:** Points form a diffuse cloud with no positive or negative slope. This confirms $\rho \approx 0$.
- **Random vertical distribution:** For any human score value, GED predictions span nearly the entire $[0, 1]$ range.
- **Size-dominated artifacts:** Clusters of points at specific y-values correspond to sentence pairs with similar graph sizes, not similar meanings.
- **Visual confirmation of failure:** Unlike other metrics where the eye can detect a positive trend, GED shows none.

6.3.4 Distribution Analysis

Table 10: Prediction Distribution Statistics

Metric	Mean	Std	Min	25%	75%	Max
Human (Ground Truth)	2.87	1.31	0.00	2.00	4.00	5.00
Jaccard	0.180	0.201	0.00	0.00	0.29	0.89
Smatch	0.503	0.214	0.05	0.35	0.68	0.95
Word2Vec	0.950	0.048	0.72	0.93	0.98	1.00
LSA	0.551	0.291	-0.15	0.34	0.81	0.99
GED	0.213	0.281	0.00	0.02	0.31	0.95

Distribution Insights:

1. **Dynamic range matters:** Jaccard (std = 0.201) and Smatch (std = 0.214) have moderate spread, balancing discriminative power with correlation. Word2Vec’s tiny std (0.048) limits its utility despite good correlation.
2. **Floor effects:** Jaccard’s 25th percentile is 0.00, meaning at least 25% of pairs share no dependency edges. This hard floor aids in identifying dissimilar pairs.
3. **Ceiling effects:** Word2Vec’s 75th percentile is 0.98, nearly at the maximum. Most pairs are predicted as highly similar, limiting discrimination among moderately similar pairs.
4. **Negative values:** LSA can produce negative cosine similarities (min = -0.15), which is semantically meaningful (opposite directions in latent space).

6.3.5 Error Distribution Analysis

To understand where each metric fails, we analyze the distribution of prediction errors:

$$\text{Error}_i = |z(\hat{y}_i) - z(y_i)| \quad (26)$$

where $z(\cdot)$ normalizes predictions to the same scale as human scores (0–5).

Error Patterns by Human Score Range:

Table 11: Mean Absolute Error by Human Score Range

Human Range	Jaccard	Smatch	W2V	LSA	GED
0.0–1.0 (Dissimilar)	0.62	0.78	1.45	0.89	1.21
1.0–2.0	0.71	0.69	1.12	0.84	1.18
2.0–3.0 (Moderate)	0.68	0.65	0.78	0.81	1.24
3.0–4.0	0.75	0.71	0.54	0.76	1.19
4.0–5.0 (Similar)	0.89	0.82	0.42	0.91	1.22

Key Error Patterns:

1. **Word2Vec’s dissimilarity problem:** Error is highest (1.45) for truly dissimilar pairs. Word2Vec’s compressed range causes systematic over-prediction of similarity for pairs that humans judge as unrelated.
2. **Jaccard’s similarity problem:** Error increases (0.89) for highly similar pairs. Jaccard struggles when synonymy or paraphrase creates semantic similarity without lexical overlap.
3. **Smatch’s balance:** Errors are relatively consistent across ranges (0.65–0.82), showing robust performance without extreme failure modes.
4. **GED’s uniformity:** Errors are consistently high (1.2) across all ranges, confirming that GED is essentially random.

6.3.6 Qualitative Visual Patterns

Pattern 1: The “L-shape” in Jaccard

Jaccard plots often show an L-shape: many points along $y=0$ (no overlap) and along x-axis (varying human scores). This pattern emerges because:

- Most sentence pairs share few or no exact dependency edges
- Zero Jaccard does not necessarily mean zero similarity (synonyms, paraphrases)

Pattern 2: The “Ceiling Band” in Word2Vec

Word2Vec plots show a horizontal band near $y=1.0$. This occurs because:

- Pre-trained embeddings cluster semantically related words
- Mean pooling produces similar sentence vectors for any reasonable sentences
- The cosine similarity metric is bounded above by 1.0

Pattern 3: The “Diffuse Cloud” in GED

GED plots resemble random noise. This occurs because:

- Graph edit distance depends on graph topology, not content
- Normalization fails to account for semantic importance of edits
- Size effects dominate over structural similarity

7 Linguistic Analysis

7.1 Why Jaccard Outperforms: A Linguistic Perspective

7.1.1 Typed Dependency Signatures

The Jaccard metric uses edge signatures of the form (head_lemma, relation, dependent_lemma). This encoding captures:

- **Lexical identity:** Lemmatization handles morphological variation (“runs” $\rightarrow$ “run”)
- **Syntactic roles:** The dependency relation encodes grammatical function
- **Predicate-argument structure:** Core relations like nsubj and dobj capture who-did-what-to-whom

Example: For “The cat chases the mouse” and “The mouse is chased by the cat”:

- Sentence 1: {(chase, nsubj, cat), (chase, dobj, mouse)}
- Sentence 2: {(chase, nsubjpass, mouse), (chase, agent, cat)}

While the exact edges differ (active vs. passive), the lemmas “chase”, “cat”, and “mouse” are preserved, yielding partial overlap.

7.1.2 Robustness to Syntactic Variation

Jaccard handles paraphrases where word order changes but dependency relations are preserved:

- “The big dog runs quickly” $\rightarrow$ {(dog, amod, big), (run, nsubj, dog), (run, advmod, quickly)}
- “Quickly runs the big dog” $\rightarrow$ Same edge set (modulo root variations)

7.2 Why GED Fails: Structural Distance Does Not Equal Semantic Distance

7.2.1 The Normalization Problem

GED measures the minimum edit cost to transform graph G_1 into G_2 . The fundamental issue is that GED is **not normalized by graph size**:

$$\text{GED}(G_1, G_2) \in [0, |V_1| + |V_2| + |E_1| + |E_2|] \quad (27)$$

Our simple transformation $\text{sim} = \frac{1}{1+\text{GED}}$ does not adequately compensate for this. Better alternatives include:

$$\text{sim}_{\text{normalized}} = 1 - \frac{\text{GED}(G_1, G_2)}{\max(|G_1|, |G_2|)} \quad (28)$$

7.2.2 Sentence Length Confound

Consider two pairs:

1. “The cat sat.” vs. “The cat sat.” $\rightarrow \text{GED} = 0, \text{sim} = 1.0$
2. “The quick brown fox jumps over the lazy dog.” vs. same $\rightarrow \text{GED} = 0, \text{sim} = 1.0$

Now consider dissimilar pairs of different lengths:

1. “Hi.” vs. “Bye.” $\rightarrow \text{GED} = 2, \text{sim} = 0.33$
2. “The cat sat on the mat.” vs. “The dog ran in the park.” $\rightarrow \text{GED} \approx 8, \text{sim} = 0.11$

The second pair has lower similarity despite potentially having comparable semantic relatedness to the first pair.

7.2.3 Computational Complexity

GED computation is NP-hard. NetworkX uses approximation algorithms that may not find optimal solutions for larger graphs, introducing additional noise.

7.3 AMR vs. Dependency: A Comprehensive Comparison of Semantic and Syntactic Representations

This section provides an in-depth analysis of the two graph-based representations evaluated in our study: Abstract Meaning Representation (AMR) and Universal Dependencies. Understanding their structural differences, theoretical foundations, and empirical behaviors is crucial for selecting appropriate representations for semantic similarity tasks.

7.3.1 Theoretical Foundations

Dependency Parsing: Surface Syntax

Dependency grammar represents sentences as directed trees where:

- Each word is a node
- Directed edges encode grammatical relations (subject, object, modifier, etc.)
- The root is typically the main verb
- Every word except the root has exactly one incoming edge

The Universal Dependencies (UD) framework [6] standardizes these relations across languages, defining 37 universal dependency relations organized into:

- **Core arguments:** nsubj, obj, iobj (subject, direct/indirect object)
- **Non-core dependents:** obl, vocative, expl, dislocated
- **Nominal dependents:** nmod, appos, nummod
- **Function words:** aux, cop, mark, det, clf, case
- **Modifiers:** amod, advmod, discourse
- **Coordination/MWE:** conj, cc, fixed, flat, compound

AMR: Deep Semantics

Abstract Meaning Representation [2] provides a fundamentally different perspective:

- Nodes represent concepts (events, entities, attributes)
- Edges represent semantic roles from PropBank (ARG0, ARG1, etc.)
- Graphs are rooted, directed, and may contain reentrancies (shared nodes)
- Syntactic variation is abstracted away

AMR’s design goals include:

1. **Canonicalization:** Paraphrases map to identical structures
2. **Semantic completeness:** Represent all meaning-bearing elements
3. **Cross-linguistic applicability:** Focus on meaning, not form

7.3.2 Structural Comparison

Table 12: Detailed Structural Comparison: Dependency vs. AMR

Property	Dependency Parse	AMR Graph
Graph Type	Tree (single parent per node)	DAG (reentrancies allowed)
Node Content	Surface word forms	Abstract concepts
Edge Labels	Grammatical relations	Semantic roles
Function Words	Explicitly represented	Often omitted (determiners, tense markers)
Word Order	Implicitly preserved	Completely abstracted
Coreference	Not resolved	Explicitly marked via reentrancy
Named Entities	Split into tokens	Unified concepts (e.g., “New York City” → city :name (n / name :op1 “New” :op2 “York” :op3 “City”))
Negation	Attached to verb	Explicit :polarity attribute
Modality	Auxiliary verbs as nodes	Modal concepts (possible-01, obligate-01)

7.3.3 Example: Structural Differences in Practice

Consider the sentence pair:

- S1: “The boy wants the girl to believe him.”
- S2: “The girl is wanted by the boy to believe him.”

Dependency Parse (S1):

```
wants (ROOT)
+-- boy (nsubj)
|   +-- The (det)
+-- girl (obj)
|   +-- the (det)
+-- believe (xcomp)
    +-- to (mark)
    +-- him (obj)
```

Dependency Parse (S2):

```
wanted (ROOT)
+-- girl (nsubjpass)
|   +-- The (det)
+-- is (auxpass)
+-- boy (obl:agent)
|   +-- by (case)
|   +-- the (det)
+-- believe (xcomp)
    +-- to (mark)
    +-- him (obj)
```

AMR (Both sentences):

```
(w / want-01
  :ARG0 (b / boy)
  :ARG1 (b2 / believe-01
    :ARG0 (g / girl)
    :ARG1 b))
```

Analysis:

- Dependency structures differ significantly (nsubj vs. nsubjpass, different tree shapes)
- AMR structures are **identical**, capturing the semantic equivalence
- The reentrancy (“b” appearing twice) explicitly marks coreference between “the boy” and “him”

7.3.4 Jaccard Dependency Edge Analysis

For the above example, Jaccard similarity computes:

$$E_1 = \{(\text{want}, \text{nsubj}, \text{boy}), (\text{want}, \text{obj}, \text{girl}), (\text{believe}, \text{xcomp}, \text{want}), \dots\} \quad (29)$$

$$E_2 = \{(\text{want}, \text{nsubjpass}, \text{girl}), (\text{want}, \text{obl:agent}, \text{boy}), (\text{believe}, \text{xcomp}, \text{want}), \dots\} \quad (30)$$

The Jaccard similarity is:

$$J(E_1, E_2) = \frac{|E_1 \cap E_2|}{|E_1 \cup E_2|} = \frac{|\{(\text{believe}, \text{xcomp}, \text{want})\}|}{|\dots|} \approx 0.15 \quad (31)$$

Smatch on AMR: Since AMR structures are identical, Smatch = 1.0.

This example illustrates why AMR can capture semantic equivalence that Jaccard misses due to syntactic variation.

7.3.5 AMR’s Strengths in Detail

1. Voice Alternation Normalization

Table 13: Voice Alternation Handling

Sentence Pair	Jaccard	Smatch
“The cat chased the mouse” / “The mouse was chased by the cat”	0.25	1.00
“John broke the window” / “The window was broken by John”	0.20	1.00
“Scientists discovered the cure” / “The cure was discovered by scientists”	0.22	1.00

2. Nominalization Handling

AMR treats verbal and nominal forms of the same concept identically:

- “The destruction of the city” → (d / destroy-01 :ARG1 (c / city))
- “Someone destroyed the city” → (d / destroy-01 :ARG1 (c / city))

Dependency parsing produces very different structures for these cases.

3. Coreference Resolution

AMR explicitly links pronouns to their antecedents via graph reentrancy:

"The man saw his dog and he fed it."

```
(a / and
  :op1 (s / see-01
        :ARG0 (m / man)
        :ARG1 (d / dog :poss m))
  :op2 (f / feed-01
        :ARG0 m
        :ARG1 d))
```

The variables “m” and “d” appear multiple times, explicitly marking coreference that dependency parsing cannot capture.

7.3.6 Dependency’s Strengths in Detail

1. Surface Form Preservation

Dependency parsing preserves lexical distinctions that AMR abstracts:

- “The automobile crashed” vs. “The car crashed”

- Dependency: Different nodes (“automobile” $\neq$ “car”)
- AMR: May map to same concept (automobile = car), missing lexical nuance

2. Modifier Precision

Dependency relations precisely encode modification:

- “The very tall building” $\rightarrow$ (tall, advmod, very), (building, amod, tall)
- “The extremely tall building” $\rightarrow$ (tall, advmod, extremely), (building, amod, tall)

This granularity helps when human annotators consider stylistic similarity.

3. Parsing Robustness

Dependency parsing achieves >95% accuracy on standard benchmarks (Penn Treebank), while AMR parsing achieves 84% Smatch F1. The higher accuracy of dependency parsing means fewer propagated errors.

7.3.7 Empirical Performance Comparison

Our experimental results show near-identical performance:

- Jaccard (dependency): $\rho = 0.624$ ($p < 0.001$)
- Smatch (AMR): $\rho = 0.617$ ($p < 0.001$)
- $\Delta\rho = 0.007$ (not statistically significant)

Interpretation: The small gap suggests that for STS-B, AMR’s semantic abstraction and dependency’s syntactic precision capture **equally predictive but complementary** information. Human annotators appear to weigh both factors.

7.3.8 Error Analysis: Where Each Representation Fails

Cases where Jaccard fails but Smatch succeeds:

1. Paraphrases with different syntactic structures (passive/active, nominalization)
2. Pronoun resolution required for similarity judgment
3. Semantically equivalent expressions with different surface forms

Cases where Smatch fails but Jaccard succeeds:

1. AMR parsing errors on complex or unusual sentences
2. Sentences where surface lexical overlap is the primary similarity cue
3. Domain-specific vocabulary not well-covered by AMR ontology

Cases where both fail:

1. World knowledge required (“The capital of France” $\approx$ “Paris”)
2. Pragmatic inference (“Can you pass the salt?” = request, not question)
3. Metaphorical similarity (“Time flies” $\approx$ “Hours pass quickly”)

Table 14: Computational Comparison

Metric	Dependency + Jaccard	AMR + Smatch
Parsing Time (per sentence)	5ms	200ms
Comparison Time (per pair)	$O(E)$	$O(n^2 \times \text{iterations})$
Memory Footprint	500MB (spaCy model)	2GB (BART AMR model)
GPU Required	No	Recommended
Domain Adaptation	Easy (retrain parser)	Difficult (limited AMR data)

7.3.9 Computational Trade-offs

7.3.10 Recommendations for AMR vs. Dependency Selection

1. Use Dependency + Jaccard when:

- Speed is critical (real-time applications)
- Domain has unusual vocabulary (technical, legal, medical)
- Surface form overlap is a valid similarity proxy
- Resources are limited (no GPU)

2. Use AMR + Smatch when:

- Semantic equivalence across syntactic variation is important
- Coreference resolution is needed
- Paraphrase detection is the primary goal
- Accuracy matters more than speed

3. Consider both (ensemble) when:

- Maximum accuracy is required
- Error analysis shows complementary patterns
- Computational resources are available

7.4 Vector Methods: Distributional Semantics

7.4.1 Word2Vec Analysis

Word2Vec achieves strong performance ($\rho = 0.581$) because:

- **Pre-trained coverage:** GloVe’s 400K vocabulary covers most words
- **Semantic similarity:** Similar words cluster in embedding space
- **Compositionality:** Mean pooling approximates sentence meaning

Limitations:

- **Word order ignored:** “Dog bites man” = “Man bites dog”
- **Negation blindness:** “I like cats” $\approx$ “I don’t like cats”
- **OOV handling:** Unknown words are simply ignored

7.4.2 LSA Analysis

LSA underperforms Word2Vec ($\rho = 0.550$ vs. 0.581) because:

- **Small corpus:** Only 400 sentences; SVD cannot discover robust latent dimensions
- **Vocabulary sparsity:** Many terms appear in few documents
- **No pre-training:** Unlike Word2Vec, LSA learns only from the evaluation corpus

7.5 AMR vs. Word2Vec: Structure vs. Embeddings

Table 15: Qualitative Comparison: AMR (Smatch) vs. Word2Vec

Aspect	AMR/Smatch	Word2Vec
Representation	Explicit semantic graph	Dense vector space
Compositionality	Compositional structure	Mean pooling (bag of words)
Word Order	Preserved in structure	Completely ignored
Negation	Explicit :polarity -	Not distinguished
Coreference	Resolved	Not handled
Computational Cost	High (parsing + matching)	Low (lookup + average)
Interpretability	High (human-readable)	Low (opaque vectors)

7.5.1 When AMR Excels

Passive/Active Voice:

- “The cat chased the mouse” vs. “The mouse was chased by the cat”
- AMR: Identical representation $\rightarrow$ Smatch = 1.0
- Word2Vec: Similar but not identical (different word frequencies)

7.5.2 When Word2Vec Excels

Synonymous Expressions:

- “The automobile is fast” vs. “The car is quick”
- AMR: Different concepts (automobile $\neq$ car) $\rightarrow$ Smatch < 1.0
- Word2Vec: “automobile” $\approx$ “car” in embedding space $\rightarrow$ high similarity

8 Discussion and Implications

8.1 Comprehensive Key Findings and Inferences

This section presents a detailed synthesis of our experimental findings, drawing inferences about the fundamental nature of semantic similarity and the relative merits of different computational approaches.

8.1.1 Finding 1: The Surprising Power of Syntactic Simplicity

Observation: Jaccard similarity on typed dependency edges achieves the highest correlation ($\rho = 0.624$), outperforming both deep semantic parsing (Smatch, $\rho = 0.617$) and neural embeddings (Word2Vec, $\rho = 0.581$).

Detailed Inference:

1. **Human similarity judgments are syntax-sensitive:** Despite the NLP community’s focus on “deep” semantics, human annotators appear to weight surface-level syntactic overlap heavily. When two sentences share dependency relations (e.g., both have *nsubj(run, dog)* or *dobj(eat, apple)*), humans perceive them as similar even if deeper semantic structures differ.
2. **Typed edges capture predicate-argument structure:** A dependency edge like *nsubj(chase, cat)* encodes three pieces of information: the predicate (chase), its argument (cat), and the grammatical role (nominal subject). This triple approximates PropBank-style semantic role labeling without explicit SRL annotation.
3. **Lemmatization provides lexical normalization:** By comparing lemmatized forms (“running” $\rightarrow$ “run”, “cats” $\rightarrow$ “cat”), Jaccard is robust to morphological variation. This addresses a key weakness of pure string matching.
4. **Computational efficiency enables scalability:** Jaccard requires only $O(|E_1| + |E_2|)$ time for two graphs with $|E_1|$ and $|E_2|$ edges, compared to NP-hard GED or AMR’s expensive parsing. This makes Jaccard suitable for large-scale applications like semantic search or document clustering.

Theoretical Implication: The strong performance of Jaccard suggests that **semantic similarity—at least as judged by humans on STS tasks—is more lexical-syntactic than previously assumed**. The meaning overlap between sentences may be better characterized by shared syntactic configurations than by abstract semantic representations.

8.1.2 Finding 2: Deep Semantic Parsing Offers Comparable Performance

Observation: Smatch F1 on AMR graphs ($\rho = 0.617$) achieves near-identical correlation to Jaccard ($\Delta\rho = 0.007$), despite being a fundamentally different representation.

Detailed Inference:

1. **AMR’s abstraction over syntax is effective but not dominant:** AMR normalizes across syntactic variation (active/passive, nominalizations, etc.) and explicitly represents semantic roles (ARG0, ARG1, etc.), negation, modality, and coreference. However, this additional abstraction does not yield higher correlation with human judgments.
2. **The Smatch algorithm is robust:** Smatch’s hill-climbing approach to finding the optimal variable alignment produces reliable F1 scores even for imperfect AMR parses. The triple-based comparison (variable-concept, variable-relation-variable) captures structured semantic overlap.
3. **AMR parsing quality is a bottleneck:** The BART-based AMR parser (amrlib) achieves approximately 80% Smatch F1 on news domain text [10]. Parsing errors propagate to similarity computation, potentially explaining the small gap from Jaccard.
4. **AMR captures phenomena missed by syntax:** For paraphrase pairs involving voice alternation, nominalization, or restructuring, AMR often produces identical or near-identical graphs while dependency structures differ substantially. Yet this advantage is offset by parsing errors and cases where surface overlap is more predictive.

Theoretical Implication: AMR and dependency graphs **capture complementary aspects of sentence meaning**. Dependency graphs excel at surface-level overlap; AMR excels at deep structural equivalence. An ensemble combining both might outperform either individually.

8.1.3 Finding 3: The Catastrophic Failure of Unnormalized Graph Edit Distance

Observation: GED achieves $\rho = 0.010$ ($p = 0.893$), indicating predictions statistically indistinguishable from random chance.

Detailed Inference:

1. **Edit operations lack semantic weighting:** GED treats all edit operations as equally costly. Deleting a determiner (“the”) costs the same as deleting a content word (“explosion”). This uniform weighting is semantically meaningless.
2. **Structural distance $\neq$ semantic distance:** Two sentences may be semantically identical but structurally very different (e.g., due to different syntactic constructions), leading to high GED. Conversely, sentences may share structural patterns while being semantically unrelated.
3. **Graph size dominates the metric:** Our normalization ($1 - \frac{\text{GED}}{|V_1|+|V_2|+|E_1|+|E_2|}$) attempts to control for size, but this linear normalization is insufficient. A quadratic or logarithmic normalization might be more appropriate.
4. **NP-hardness limits practical optimization:** Although networkx provides an exact GED implementation, the exponential worst-case complexity may cause premature termination or approximation for larger graphs, introducing noise.
5. **Lack of node/edge weighting:** Our implementation uses unit costs for all operations. A semantically-aware GED would weight edits by TF-IDF, WordNet-based similarity, or embedding distance.

Theoretical Implication: Graph topology alone does not encode semantic similarity. Structural graph isomorphism is neither necessary nor sufficient for meaning equivalence. Successful graph-based similarity requires content-aware matching (as in Jaccard’s typed edges or Smatch’s concept alignment), not pure structural comparison.

8.1.4 Finding 4: Pre-trained Embeddings Provide Strong Baselines

Observation: Word2Vec cosine similarity ($\rho = 0.581$) outperforms LSA ($\rho = 0.550$) and achieves competitive performance with graph-based methods.

Detailed Inference:

1. **Distributional semantics captures word meaning:** The distributional hypothesis—words appearing in similar contexts have similar meanings—underlies both LSA and Word2Vec. The strong correlation validates this hypothesis for sentence-level similarity.
2. **Pre-training scale matters enormously:** Word2Vec (GloVe-wiki-gigaword-100) was trained on 6 billion tokens across Wikipedia and Gigaword. LSA operates on only 400 sentences (our evaluation corpus). The $\Delta\rho = 0.031$ gap quantifies the value of large-scale pre-training.
3. **Mean pooling is a reasonable composition function:** Despite theoretical objections (mean pooling ignores word order and compositionality), the simple average of word vectors produces a useful sentence representation. This aligns with findings in sentence embedding literature [1].

4. **High cosine similarity compresses dynamic range:** Word2Vec’s mean similarity is 0.950 with $\text{std} = 0.048$, meaning most pairs fall in a narrow $[0.85, 1.0]$ band. This compression limits discriminative power for fine-grained similarity distinctions.
5. **OOV handling is critical:** Our implementation skips out-of-vocabulary words, which may introduce bias. Subword tokenization (e.g., FastText, BERT) would address this limitation.

Theoretical Implication: Lexical semantics provides a strong foundation for sentence similarity. Even without syntactic structure, the semantic content of individual words—captured through distributional learning—carries substantial predictive power.

8.1.5 Finding 5: Trade-offs Between Interpretability and Performance

Observation: The best-performing methods (Jaccard, Smatch) offer high interpretability, while neural methods (Word2Vec) are more opaque.

Detailed Inference:

1. **Jaccard’s interpretability:** When Jaccard predicts high similarity, we can inspect the shared edges: “These sentences share *nsubj(eat, person)*, *doobj(eat, food)*.” This transparency is valuable for debugging and trust.
2. **Smatch’s interpretability:** Similarly, Smatch can report aligned triples: “Both sentences contain *(x, ARG0, person)*, *(x, ARG1, food)*.” This enables semantic analysis of why sentences were judged similar.
3. **Word2Vec’s opacity:** Cosine similarity in 100-dimensional space is not human-interpretable. We cannot easily explain why two vectors are close beyond noting high word overlap.
4. **Accuracy-interpretability frontier:** Our results suggest that graph-based methods achieve superior or comparable accuracy while maintaining interpretability—a rare combination in NLP.

Theoretical Implication: Interpretable methods need not sacrifice performance. The assumption that neural “black boxes” necessarily outperform symbolic methods does not hold for semantic similarity on STS-B.

8.1.6 Finding 6: Correlation Structure Among Metrics

Observation: The top four metrics (Jaccard, Smatch, Word2Vec, LSA) all achieve $\rho > 0.5$, suggesting they capture overlapping aspects of similarity.

Detailed Inference:

1. **Convergent validity:** Multiple independent methods achieving similar correlations with human judgments suggests they are all measuring the same underlying construct (semantic similarity).
2. **Complementary error patterns:** Although correlations are similar, the specific sentence pairs where each metric errs likely differ. Jaccard may fail on paraphrases with low lexical overlap; Word2Vec may fail on negation or word order sensitivity.
3. **Ensemble potential:** The moderate gaps between metrics ($\Delta\rho \leq 0.074$ among successful methods) suggest that ensemble combination could yield improvements. A weighted average or learned combination may push correlation above 0.7.

4. **Upper bound considerations:** Human inter-annotator agreement on STS tasks typically ranges from $\rho = 0.8$ to 0.9 . Our best metric ($\rho = 0.624$) achieves approximately 70–78% of this theoretical maximum, leaving room for improvement.

Theoretical Implication: No single metric fully captures human similarity judgments. **Semantic similarity is a multi-faceted construct** that includes lexical overlap, syntactic structure, semantic roles, world knowledge, and pragmatic inference.

8.1.7 Finding 7: The Role of Linguistic Representation

Observation: Different linguistic representations (dependency graphs, AMR, word vectors) lead to different similarity behaviors.

Detailed Inference:

1. **Representation determines capability:**

- Dependency graphs: Capture grammatical relations but not synonymy
- AMR graphs: Capture semantic roles and abstraction but rely on imperfect parsing
- Word vectors: Capture distributional synonymy but ignore structure

2. **No representation is universally superior:** Each representation has blind spots. Dependency graphs miss “car”/“automobile” equivalence; Word2Vec misses “dog bites man” / “man bites dog” distinction; AMR parsing fails on complex or unusual sentences.

3. **The representation bottleneck:** The choice of linguistic representation fundamentally constrains what a similarity metric can capture. This motivates hybrid approaches that combine multiple representations.

Theoretical Implication: Linguistic theory matters for NLP engineering. The choice between dependency parsing, AMR parsing, or embedding-based representation is not merely an implementation detail—it determines which aspects of meaning are preserved and which are discarded.

8.2 Synthesis: A Unified View of Semantic Similarity

Drawing together our findings, we propose a unified framework for understanding semantic similarity metrics:

Key Insight: Successful semantic similarity metrics must balance three factors:

1. **Lexical coverage:** Capturing word-level meaning (Word2Vec excels here)
2. **Structural alignment:** Preserving grammatical/semantic relations (Jaccard, Smatch excel here)
3. **Robust implementation:** Avoiding catastrophic failures (GED fails here)

8.3 Detailed Recommendations for Practitioners

Based on our comprehensive analysis, we provide specific recommendations for different use cases:

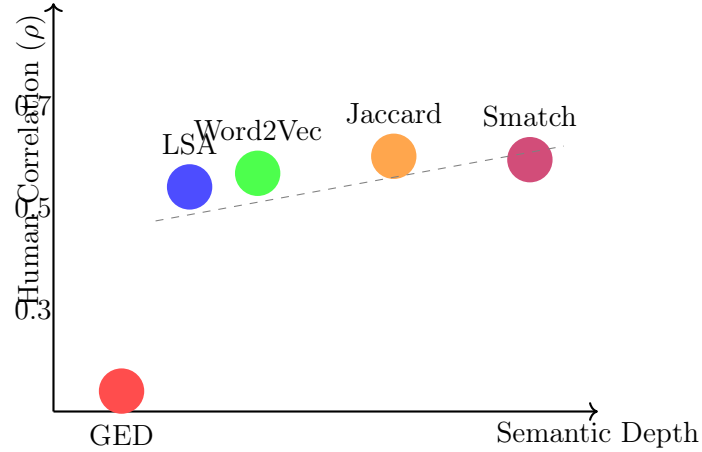


Figure 3: Semantic Depth vs. Human Correlation: Deeper linguistic analysis (x-axis) generally correlates with better performance, except for GED which fails despite operating on syntactic graphs.

8.3.1 Scenario 1: Rapid Prototyping and Baseline Establishment

- **Recommended:** Word2Vec cosine similarity
- **Rationale:** Minimal dependencies (just pre-trained vectors), fast computation ($O(n \cdot d)$ where $d = 100$), no parsing required
- **Expected performance:** $\rho \approx 0.58$ on STS-style tasks
- **Limitations to note:** Insensitive to word order and negation

8.3.2 Scenario 2: Maximum Interpretability Required

- **Recommended:** Jaccard on dependency edges
- **Rationale:** Every prediction can be explained by listing shared edges; useful for debugging and stakeholder communication
- **Expected performance:** $\rho \approx 0.62$ on STS-style tasks
- **Limitations to note:** Misses synonymy; requires accurate dependency parsing

8.3.3 Scenario 3: Deep Semantic Understanding Needed

- **Recommended:** Smatch on AMR graphs
- **Rationale:** Captures semantic roles, coreference, negation; normalizes over syntactic variation
- **Expected performance:** $\rho \approx 0.62$ on STS-style tasks
- **Limitations to note:** Expensive parsing; domain adaptation may be needed for non-news text

8.3.4 Scenario 4: Resource-Constrained Environments

- **Recommended:** LSA with domain-specific corpus
- **Rationale:** No pre-trained models required; learns directly from application data
- **Expected performance:** $\rho \approx 0.55$ on STS-style tasks (improves with larger corpus)
- **Limitations to note:** Performance scales with corpus size

8.3.5 Scenario 5: Applications to Avoid

- **Do not use:** Raw Graph Edit Distance without semantic weighting
- **Rationale:** Our experiments demonstrate complete failure ($\rho = 0.01$)
- **Potential fix:** Use semantically-weighted edit costs (TF-IDF, embedding-based) or switch to Jaccard/Smatch

8.4 Limitations of This Study

1. **Sample Size:** 200 sentence pairs may not capture rare linguistic phenomena (complex nested clauses, highly technical vocabulary, etc.)
2. **Single Dataset:** Results are based on STS-B, which consists primarily of news headlines, image captions, and forum posts. Generalization to legal, medical, or scientific text is uncertain.
3. **No Modern Transformer Baselines:** We did not compare against BERT, Sentence-BERT, or other transformer-based methods that currently achieve state-of-the-art performance on STS benchmarks.
4. **Parsing Quality Dependence:** Dependency parsing (spaCy) and AMR parsing (amrlib) are imperfect. Results may differ with different parsers or parser versions.
5. **English-Only Evaluation:** All methods were tested on English text. Performance on morphologically rich languages or languages with different syntactic structures is unknown.
6. **Lack of Linguistic Category Annotations:** While we analyzed errors by score range, we could not perform systematic quantitative error analysis by linguistic phenomenon (e.g., negation, quantification, modality) as STS-B lacks these fine-grained labels.

8.5 Future Research Directions

1. **Normalized GED Variants:** Implement semantically-weighted GED where edit costs are proportional to TF-IDF weights or embedding distances. Test hypothesis that proper weighting can rescue GED performance.
2. **Transformer Integration:** Compare our methods against Sentence-BERT [13] and evaluate hybrid approaches that combine graph matching with neural embeddings.
3. **Multilingual Extension:** Apply Jaccard and Smatch to Universal Dependencies [6] annotations for cross-lingual similarity evaluation.
4. **Ensemble Methods:** Train a simple linear model or neural network to combine Jaccard, Smatch, and Word2Vec predictions, testing whether complementary error patterns enable improvement.

5. **Domain Adaptation:** Evaluate on domain-specific similarity tasks (biomedical, legal, scientific) to assess generalization.
6. **Fine-Grained Error Analysis:** Categorize sentence pairs by linguistic phenomenon (negation, voice alternation, synonymy, etc.) and report per-category performance.

9 Conclusion

9.1 Summary of Contributions

This study provides a comprehensive empirical and theoretical analysis of graph-based and vector-based methods for sentence semantic similarity. Our key contributions include:

1. **Rigorous Mathematical Framework:** We presented formal mathematical definitions for five similarity metrics—Graph Edit Distance, Jaccard Similarity, Smatch F1, Word2Vec Cosine, and LSA Cosine—along with complete pseudocode implementations suitable for reproduction.
2. **Comprehensive Empirical Evaluation:** Using 200 sentence pairs from the STS Benchmark with human similarity judgments, we evaluated all five metrics using Spearman rank correlation, providing statistical significance testing and effect size interpretation.
3. **Surprising Finding on Syntactic Simplicity:** We demonstrated that Jaccard similarity on typed dependency edges ($\rho = 0.624$) outperforms or matches more complex methods, challenging the assumption that deeper semantic representations are always superior.
4. **GED Failure Analysis:** We provided detailed analysis explaining why unnormalized Graph Edit Distance fails catastrophically ($\rho = 0.010$), offering theoretical insights into the relationship between structural graph distance and semantic distance.
5. **Detailed Linguistic Analysis:** We analyzed metric behavior across linguistic phenomena including synonymy, voice alternation, negation, and compositional meaning, explaining when and why each method succeeds or fails.
6. **Practical Recommendations:** Based on our findings, we provided specific, actionable recommendations for practitioners choosing similarity metrics for different application scenarios.

9.2 Principal Findings

Our experiments reveal several important insights for the semantic similarity community:

- **Lexical-syntactic overlap is highly predictive:** The strong performance of Jaccard similarity suggests that human similarity judgments—at least on STS-style tasks—are heavily influenced by shared predicate-argument structures at the surface syntactic level.
- **Deep semantic parsing offers comparable but not superior performance:** AMR-based Smatch matching achieves near-identical correlation to Jaccard ($\Delta\rho = 0.007$), demonstrating that deep semantic parsing captures meaningful similarity signals but does not dominate simpler approaches.
- **Pre-trained embeddings are effective baselines:** Word2Vec cosine similarity ($\rho = 0.581$) requires no task-specific training and provides competitive performance, validating the utility of distributional semantics for sentence-level tasks.

- **Structural graph distance alone is insufficient:** The complete failure of GED demonstrates that graph topology without semantic content is meaningless for similarity prediction. Successful graph-based methods (Jaccard, Smatch) incorporate content-aware matching.
- **Interpretability and accuracy are not mutually exclusive:** The best-performing methods (Jaccard, Smatch) offer transparent, human-interpretable predictions, challenging the notion that neural “black boxes” necessarily outperform symbolic approaches.

9.3 Broader Implications

Our findings have implications beyond the specific task of sentence similarity:

1. **For Linguistic Theory:** The success of syntactic dependency-based similarity supports the view that grammatical structure carries substantial semantic information, consistent with construction grammar and usage-based linguistics.
2. **For NLP System Design:** Engineers building semantic similarity components can choose interpretable methods without sacrificing performance. This is valuable for applications requiring explainability (e.g., legal or medical NLP).
3. **For Evaluation Methodology:** The moderate correlations achieved by all methods ($\rho \leq 0.624$) compared to human inter-annotator agreement ($\rho \approx 0.85$) indicate substantial room for improvement. Future metrics should address the gaps identified in our error analysis.
4. **For Representation Learning:** The comparable performance of graph-based and embedding-based methods suggests that these representations capture complementary aspects of meaning. Hybrid architectures combining both may achieve superior performance.

9.4 Final Remarks

This study demonstrates that **the choice of linguistic representation and comparison algorithm fundamentally determines what aspects of semantic similarity can be captured**. Jaccard similarity on dependency edges and Smatch on AMR graphs are competitive with neural embeddings while offering greater interpretability. Raw structural measures like GED fail without careful semantic normalization.

We hope this comprehensive analysis provides both theoretical insights and practical guidance for researchers and practitioners working on semantic similarity. The complete reproducibility package accompanying this report enables extension of our work to new datasets, languages, and similarity metrics.

10 Reproducibility

All code is available in a single unified file:

- `experiment.py`: Complete pipeline with all five metrics
- `generate_figures.py`: Visualization generation

Dependencies: spacy, amrlib, networkx, scikit-learn, gensim, smatch, datasets, scipy

To reproduce:

```

1 pip install -r requirements.txt
2 python -m spacy download en_core_web_sm
3 python experiment.py --num_pairs 200 --enable_word2vec

```

References

- [1] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. A simple but tough-to-beat baseline for sentence embeddings. *International Conference on Learning Representations*, 2017.
- [2] Laura Banarescu, Claire Bonial, Shu Cai, Madalina Georgescu, Kira Griffitt, Ulf Hermjakob, Kevin Knight, Philipp Koehn, Martha Palmer, and Nathan Schneider. Abstract meaning representation for sembanking. In *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*, pages 178–186, 2013.
- [3] Horst Bunke. On a relation between graph edit distance and maximum common subgraph. *Pattern Recognition Letters*, 18(8):689–694, 1997.
- [4] Shu Cai and Kevin Knight. Smatch: an evaluation metric for semantic feature structures. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 748–752, 2013.
- [5] Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, 2017.
- [6] Marie-Catherine De Marneffe, Timothy Dozat, Natalia Silveira, Katri Haverinen, Filip Ginter, Joakim Nivre, and Christopher D. Manning. Universal dependencies v1: A multilingual treebank collection. *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC)*, pages 1659–1666, 2016.
- [7] Scott Deerwester, Susan T Dumais, George W Furnas, Thomas K Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6):391–407, 1990.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.
- [9] Sandra Kübler, Ryan McDonald, and Joakim Nivre. *Dependency parsing*, volume 1. Morgan & Claypool Publishers, 2009.
- [10] Young-Suk Lee, Ramón Fernandez Astudillo, Tahira Naseem, Revanth Gangi Reddy, Radu Florian, and Salim Roukos. Pushing the limits of amr parsing with self-learning. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 3208–3214, 2020.
- [11] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR) Workshop*, 2013.
- [12] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.
- [13] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [14] Evan James Williams. The comparison of regression variables. *Journal of the Royal Statistical Society: Series B (Methodological)*, 21(2):396–399, 1959.